Computer Standards & Interfaces 95 (2026) 104042

Contents lists available at ScienceDirect

Computer Standards & Interfaces

journal homepage: www.elsevier.com/locate/csi

Multiuser data integrity auditing atop blockchain with secure user revocation for cognitive IoT networks

Min Xie [a], Yong Yu [b,*], Ruonan Chen [c], Yanqi Zhao [d], Jianting Ning [e], Xiaoyi Yang [d], Zoe L. Jiang [a,f,*]

[a] School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen, Shenzhen, 518055, Guangdong, China
[b] School of Computer Science, Shaanxi Normal University, Xi'an, 710062, Shaanxi, China
[c] School of Cyber Science and Technology, Beihang University, Beijing, 100191, China
[d] School of Cyberspace Security, Xi'an University of Posts and Telecommunications, Xi'an, 710121, Shaanxi, China
[e] School of Cyber Science and Engineering, Wuhan University, Wuhan, 430072, Hubei, China
[f] Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies, Shenzhen, 518055, Guangdong, China

ARTICLE INFO

Keywords:
Shared data
Decentralized cloud storage
Integrity auditing
Public auditing
User revocation
Cognitive IoT

ABSTRACT

Cognitive computing over big data has advanced the cognitive Internet of Things (IoT), enhancing adaptive decision-making by the analysis of shared data, while posing challenges in the storage and multi-user sharing of large-scale real-time data. Decentralized cloud storage is a promising solution to reduce latency and prevent single-point failures, but a key factor in preventing erroneous decisions lies in the integrity of shared data. However, integrity auditing in decentralized and shared storage typically involves linear overheads, with revoked users potentially colluding with cloud providers to evade audits. To address these issues, we propose a blockchain-based multiuser data integrity auditing protocol for cognitive IoT, supporting secure user revocation and batch auditing in decentralized storage. Our protocol classifies group users into different categories and manages group users efficiently and dynamically. Besides, files stored across various storage nodes can be audited in batches and effectively updated based on novel block tags. Smart contracts deployed on the blockchain ensure fairness among participants. We formally prove the security of the protocol in the random oracle model and the algebraic group model, in particular, against collusion attacks. Finally, we evaluate the cost separately for on-chain operations and off-chain operations to show its practicality.

1. Introduction

The Internet of Things (IoT) represents a burgeoning paradigm leveraging smart devices and internet capabilities within the realm of big data, offering innovative applications for intelligent services in areas including ecological protection, smart homes, logistics, and transportation [1]. With the development of cognitive computing, the cognitive IoT has been a pivotal development [2], which integrates advanced technologies such as cognitive radio networks, endowing IoT devices with sophisticated cognitive abilities like dynamic spectrum access, environmental adaptability, and autonomous learning [2,3].

The burgeoning number of end devices within the cognitive IoT ecosystem has resulted in a significant volume of data and service requests.[1] These are crucial for adaptive decision-making, routing optimization, traffic management, and efficient resource allocation within the network. Therefore, efficiently handling and analyzing the gathered real-time data has become a pressing issue [4]. Cloud computing offers a solution, providing unlimited computational power and storage capacity to end-users, who enables users to pay solely for the services utilized, independent of local hardware. By facilitating rapid data processing and analysis through cloud servers, cognitive IoT can adjust network routing and behavior based on responsive analytics, thereby significantly augmenting resource utilization. In particular, collaborative cloud services are captured by cognitive IoT, facilitating data sharing across a wide range of users [5,6]. Indeed, inadequate data sharing can lead to decisions based on incomplete or skewed information, reducing the accuracy and efficiency of these decisions. Moreover, the lack of shared data may also lead to suboptimal resource utilization, as the network cannot adjust its behavior and routing based on comprehensive data feedback.

Similar to cloud storage, the overriding concern for shared data is data integrity. If the data for cognitive IoT is damaged in the cloud, it can affect the decision-making of other IoT devices and even the

* Corresponding authors.
E-mail addresses: yuyong@snnu.edu.cn (Y. Yu), zoeljiang@hit.edu.cn (Z.L. Jiang).

[1] https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html.

https://doi.org/10.1016/j.csi.2025.104042
Received 16 January 2025; Received in revised form 19 June 2025; Accepted 24 June 2025
Available online 7 July 2025
0920-5489/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

entire communication network [4]. Although there exist amounts of schemes [7–15] proposed by researchers to verify the integrity of cloud data, those proposals mainly focus on a single user, rather than multiple users. The outsourcing data integrity auditing protocol for multi-user is essential when multiple users need to share the data in a group. Unlike cloud data owned by a single individual, the shared aspects of outsourced data pose greater challenges for desirable multiuser data integrity auditing protocols.

In outsourcing storage applications with data sharing, users are allowed to access and even update the data within a group setting, leading to complex integrity auditing due to data tags from different users. Moreover, dynamic groups allow users to join or leave groups at any time, which is challenging. Specifically, when a user is revoked, he will be restricted from accessing and modifying the data, and his private key and corresponding tags are revoked as well. A trivial solution involves the data owner downloading all tags provided by revoked users, recalculating them, and then uploading them to the cloud server. However, this approach introduces considerable computational and communication costs. To overcome the above challenges, a series of protocols have been offered for different attractive features, including public auditing [16,17], data dynamic update auditing [18–20], group dynamic operation [21,22], privacy-preserving auditing [23,24].

In addition, the revoked users may collude with cloud service providers for profit, which seriously damages the shared cloud storage. Jiang et al. [16] introduced a method of revoking users by using group signatures, to prevent collusion between cloud providers and revoked users. However, the high computational and auditing costs associated with group signatures render this approach inefficient. More importantly, their protocol has not been formally proven to be collusion resistant. Rabaninejad et al. [22] provided a lightweight auditing protocol to achieve anti-collusion by proxy re-signature in shared cloud environments. However, all of the above protocols focus on centralized cloud storage, making them susceptible to single points of failure, in particular, unsuitable for distributed, dynamic, and ever-changing deployment landscape of cognitive IoT networks [25]. Tian et al. [26] introduced a multi-copy data sharing auditing approach, incorporating decentralized trust management to eliminate the risk of a single point of failure. However, they focused on efficient auditing of multiple replicas without considering secure user revocation of shared data. An efficient shared decentralized storage auditing protocol is expected for cognitive IoT networks to support efficient public auditing, dynamic update auditing, group dynamic operation, and collusion resistance.

1.1. Our contributions

To address the aforementioned needs in cognitive IoT networks, this paper presents a blockchain-based multiuser data integrity auditing protocol. Our protocol offers a secure and efficient solution for shared and decentralized storage integrity. Our contributions are summarized as follows.

1. Our proposal audits the integrity of shared data in decentralized storage for cognitive IoT networks while providing dynamic data updates and secure user revocation, which are key aspects of multi-user data auditing. Specifically, we introduce a novel multi-user data integrity auditing model for solving the issues of single point of failure, collusion attacks, and malicious tampering. Our proposal classifies the group users into different categories and formalizes the security model for this new auditing model. Especially, in response to the collusion between the revoked users and the storage providers, our protocol is designed to resist this type of attack.
2. To overcome the linear expansion flaw of auditing in decentralized storage, we design a new tag for file blocks, enabling batch auditing of the responses from different storage nodes, along with blockless verification, publicly verification, and dynamic updatable. In addition, we utilize the smart contracts for public verification and take advantage of the deposit atop blockchain to provide the fairness for all participants.
3. We provide a secure and efficient management support group dynamics. Attractively, a verifiable group user update method is proposed, enabling efficient and reliable user revocation, which is important for cognitive IoT networks with large end devices.
4. We provide the security proof of the protocol, including the completeness and soundness, and conduct an analysis of the fairness among all participants. We deploy off-chain and on-chain prototype implementations to evaluate their performance. Experimental results demonstrate that for a 15.6 MB file split into 1000 blocks with 256 sectors per block, requires only 84.14 ms for off-chain verification with pre-computation, or 639,723 gas for on-chain public verification. These results highlight the scheme's efficiency and suitability for lightweight IoT devices.

1.2. Related work

Cloud storage has undeniably brought great convenience to individuals and enterprises. However, the rise of various cloud security incidents has raised concerns among users. In particular, the integrity of cloud data has become a primary concern. The straightforward approach involves downloading the entire dataset for verification, which is inefficient. Moreover, users usually delete their local copies of data after outsourcing to save local space. Ateniese et al. [7] introduced the primitive of Provable Data Possession (PDP), which uses a sampling strategy to check data integrity. In a case of outsourcing 10,000 data blocks, if 1% of these blocks are lost, and you want to detect it with 99% probability, you only need to challenge up to 460 blocks, which is really efficient. Juels et al. [8] proposed a related concept known as Proof of Retrievability (PoR), employing error-correcting codes to maintain that outsourced data can be reliably retrieved. Then the Compact Proof of Retrievability (CPoR) is provided by Shacham et al. [9], which is very efficient and compact. They gave two constructions of CPoR, one for private verification, and another for public verification.

Later, dynamic operations were considered in outsourcing storage for data updating [10–12]. The origin PDP protocol cannot meet the requirements of dynamic updating because it incorporates the data block index into the tag calculation. As a result, inserting a block will cause subsequent tags to be recalculated. Ateniese et al. [10] introduced a dynamic PDP protocol. However, their protocol is limited to supporting only partial dynamics. To address the issue, Erways et al. [11] proposed a full dynamic PDP protocol by utilizing Authenticated Data Structure (ADS) and rank-based skip list. However, the verification path they used in the scheme is too long, and the computational cost and communication overhead are large. Wang et al. [12] proposed a dynamic PDP protocol, which is using Merkle hash tree. To enhance efficiency and support data dynamics in PDP, Yan et al. [27] proposed a protocol based on homomorphic hash functions, featuring an optimized operation record table for efficient block update tracking. So far, most of the dynamic data storage schemes focus on the PDP protocol. Dynamic operations cannot apply to PoR protocol directly in the sense that updating some data block needs to re-encode the entire file, which is very inefficient.

With the rise of data sharing, outsourcing storage for multi-user scenario has attracted widespread attention [16,18,19,28–34]. Considering the identity privacy disclosure of group users in integrity auditing, Wang et al. [35] designed homomorphic authenticators based on group signatures, enabling user identity privacy during auditing. To solve user revocation in shared cloud environments, Wang et al. [19] provided the shared data auditing protocol by utilizing proxy re-signatures. However, Yu et al. [36] pointed out a flaw in their protocol, noting that it fails to ensure soundness. Specifically, a malicious server could fabricate a proof, convincing the auditor that the file is intact, even

Table 1
Function comparison of some popular auditing schemes.

Scheme	Data dynamics	Collusion resistant	User revocation	Decentralized storage	Public auditing	Decentralized auditor
[18]	○	○	●	○	●	○
[16]	●	●	●	○	●	○
[29]	●	●	●	○	●	○
[22]	●	●	●	○	●	○
[32]	●	○	●	○	●	○
[26]	○	○	●	●	●	○
[33]	○	●	●	○	●	○
[34]	○	●	○	○	●	●
Ours	●	●	●	●	●	●

if it has not been properly stored. Yuan et al. [18] introduced an efficient cloud data auditing scheme to support users updating shared data and effectively revoking users using polynomial commitment. However, their scheme has been shown in [28] to be unable to resist collusion attacks, where the server colludes with the revoked users to tamper with outsourced files or even fool auditors. To prevent collusion attacks, Jiang et al. [16] developed a new tag by group signatures and proposed a secure revocation method that is resistant to collusion. However, no formal proof was provided, and the tag generation and audit operations are costly in their scheme. He et al. [29] proposed PRAYS, a dynamic group-oriented auditing scheme that supports fully dynamic operations and constant verification metadata, by introducing a blockless Merkle tree and permission-based signature. To eliminate the certificate management overhead of the schemes based on Public Key Infrastructure (PKI) and avoid the key escrow issue in identity-based schemes, Li et al. [37] proposed a certificateless shared data auditing protocol that supports public auditing and efficient user revocation in group users. Considering the limited computational resources of the users, Rabaninejad et al. [22] introduced a lightweight public shared data auditing protocol to support low computational effort of the users, identity privacy, and secure revocation. However, in their protocol, the user identities are often known to the cloud server or group manager. To enhance participant privacy, Ding et al. [32] proposed an anonymous integrity verification mechanism based on attribute-based signatures, achieving both data integrity auditing and identity protection in shared cloud environments. More recently, Tian et al. [34] focused on integrating smart contracts into the data integrity auditing process, leveraging on-chain logic to ensure audit transparency and traceability.

All the protocols mentioned above are based on centralized cloud storage, which is vulnerable to single points of failure. As distributed computing advances, decentralized storage has emerged as a prominent research focus [20,26,38–40]. Among them, Li et al. [39] proposed an identity-based PDP scheme for multi-copy data across multiple cloud servers, using homomorphic tags to enable efficient and secure integrity verification without relying on PKI. Tian et al. [26] introduced an identity-based multi-copy data sharing auditing scheme, integrating decentralized trust management to mitigate single-point failure risks and enhance group behavior control. However, their work primarily focuses on multi-copy auditing without emphasizing secure user revocation of shared data. In Table 1, we compare the related works with the work in this paper from a functional perspective.

1.3. Our organization

The remainder of the paper is organized as follows. Section 2 recalls the related techniques. Section 3 builds the system framework and security model for our protocol. We give the detailed construction in Section 4. Section 5 formalizes the security proofs, including the completeness proofs, the soundness proof and the fairness analysis. Section 6 presents the performance tests, and Section 7 concludes the paper.

2. Preliminaries

2.1. Shamir secret sharing

Here we review the Shamir secret sharing scheme, which is a really practical tool in cryptography. The scheme distributes secret shares using a polynomial $f(x)$, where the constant term represents the secret S. The secret S can be uniquely determined by Lagrange interpolation when the number of participants in the authorized set reaches the threshold. The process is detailed below.

- **Setup.** Let $GF(q)$ denote a finite filed of prime order q. The distributor chooses n different elements $x_1, x_2, \ldots, x_n$ from $GF(q)$, and distributes these elements to participants $P_i (i = 1, 2, \ldots, n)$.
- **Secret Distribution.** Let the secret value shared among participants be S. The distributor selects $t-1$ elements $a_1, a_2, \ldots, a_{t-1}$ from $GF(q)$ randomly, and construct a polynomial: $f(x) = S + \sum_{j=1}^{t-1} a_j x^j$. Then the distributor computes $s_i = f(x_i)(1 \le i \le n)$ as the secret share to the participant P_i.
- **Secret Reconstruction.** When there are any t of participants in the authorized set, they can reconstruct the secret S. if the participants are fewer than t, the secret value still remains unknown. Given t points $\{(x_1, f(x_1)), \ldots, (x_t, f(x_t))\}$, the distributor is able to reconstruct this polynomial $f(x)$ via the Lagrange interpolation theorem as $f(x) = \sum_{i=1}^{t} f(x_i)\lambda_i(x)$, where $\lambda_i(x) = \prod_{j=1, j\neq i}^{t} \frac{x-x_j}{x_i-x_j}$. Then the secret value S is $f(0)$.

2.2. Vector commitment

Vector commitment [41] enables committing to a vector $(m_1, \ldots, m_q)$, and later one can be opened at any specific position i, providing proof that m_i corresponds to the ith value in the committed sequence. Vector commitment satisfies *binding* and *hinding* as normal commitments, and also satisfies the *position binding* additionally. The *position binding* property ensures that an adversary cannot reveal two distinct values m_i and m_i' $(m_i \neq m_i')$ for the same index i in the commitment. A vector commitment is recalled below.

- **Setup.** Given a security parameter λ and a length q of a committed vector, it returns the public parameter pp. This parameter serves as the input for all subsequent algorithms.
- **Commit.** Given a message sequence $m_1, \ldots, m_q$ of size q, it generates a commitment C along with corresponding auxiliary information aux.
- **Open.** Given a message m, a position i and an auxiliary information aux, it generates a proof Λ_i to show that m is the ith component of the committed vector.
- **Verify.** On input a commitment C, a message m, a position i and a proof Λ_i, the algorithm returns *accept* only if Λ_i is valid, indicating C corresponds to a committed vector $m_1, \ldots, m_q$ where $m = m_i$. Otherwise, it outputs reject.

- **Update.** On input a commitment C, an old message m, a new message m' and the position i, this algorithm updates the message at position i from m to m'. Then this algorithm returns the updated commitment C' along with corresponding update information upd.
- **Proofupdate.** The algorithm takes a commitment C, the message m', a position i, a proof Λ_j and an update information upd as inputs, and updates the old proof Λ_j as Λ'_j, ensuring that Λ'_j remains valid for the updated commitment C'.

2.3. Weak bilinear Diffie–Hellman exponent assumption

We recall the weak bilinear Diffie–Hellman exponent assumption (l-wBDHE) which is inspired by [41]. Let G_1, G_2, and G_T be groups of prime order p with a bilinear pairing $e : G_1 \times G_2 \to G_T$. Let g_1, g_2, and $g_T = e(g_1, g_2)$ be generators of G_1, G_2, and G_T, respectively. Given $g_1^{\alpha}, g_1^{(\alpha^2)}, \ldots, g_1^{(\alpha^l)}, g_1^{(\alpha^{l+2})}, \ldots, g_1^{(\alpha^{3l})}, g_2^{\alpha}, g_2^{(\alpha^2)}, \ldots, g_2^{(\alpha^l)}$ as inputs, where $\alpha \xleftarrow{R} Z_p$. The weak bilinear Diffie–Hellman exponent problem requires to output $g_1^{(\alpha^{l+1})}$. Let $\mathbf{Adv}_{l-wBDHE}(\mathcal{A})$ be the advantage of an adversary $\mathcal{A}$ in solving the l-wBDHE problem. The l-wBDHE assumption holds if $\mathbf{Adv}_{l-wBDHE}(\mathcal{A})$ is negligible.

2.4. Rank-based Merkle hash tree

The Rank-Based Merkle Hash Tree (RMHT) [42] extends the traditional Merkle tree by embedding a rank value r_v at each internal node v, where r_v denotes the number of leaf nodes in the subtree rooted at v. Let v_l and v_r be the left and right children of v, then $r_v = r_{v_l} + r_{v_r}$. The hash value of an internal node is computed as $h_v = h(r_v \| h_{v_l} \| h_{v_r})$, where $\|$ denotes concatenation. The inclusion of r_v ensures that the position of each node in the tree is verifiable and unambiguously tied to its structural location, thus supporting both content integrity and positional correctness.

To verify a data block σ_j, the verifier uses an Auxiliary Authentication Information (AAI) structure, denoted Ω_{σ_j}, which contains the set of sibling hashes along the path from the leaf node σ_j to the root. Alongside, the position value P_{σ_j} records the exact index of σ_j among all leaf nodes. During verification, the verifier first checks that $P_{\sigma_j} = j$, ensuring the correct index, then recomputes the root hash $h_R^* = h(\sigma_j, \Omega_{\sigma_j})$, and verifies that $h_R^* = h_R$. If both conditions hold, the data block is deemed authentic and correctly positioned.

2.5. Random Oracle Model and Algebraic Group Model

Random Oracle Model. In the Random Oracle Model [43], hash functions are modeled as truly random functions that respond to each unique query with a uniformly random output, while maintaining consistency across repeated inputs. This abstraction allows for capturing the behavior of cryptographic hash functions in security proofs.

Algebraic Group Model. The Algebraic Group Model [44] is used to reason about adversaries in structured algebraic settings, especially when working with bilinear groups. It assumes that the adversary can manipulate group elements using their explicit representations, but must express any new element $Z \in \mathbb{G}_1$ it outputs as an algebraic combination of previously obtained elements, i.e., $Z = \prod_{i=1}^{N} X_i^{z_i}$, where $X_i \in \mathbb{G}_1$ are known group elements and $z_i \in \mathbb{Z}_p$.

3. System framework and security model

3.1. System framework

Our system framework contains four layers: sensing, communication, cognitive decision, and blockchain, which is described below and illustrated in Fig. 1.

The sensing layer contains numerous IoT devices, forming IoT networks such as Vehicular Ad-hoc Networks (VANET). These devices collect and sense real-time data, transmitting it to cloud servers via communication infrastructure. Simply, these devices and sensor nodes are referred to as "users". More importantly, the users in the protocol are in the form of groups, allowing members of the same group to share the cloud data. The data owner U_0 in a group authorizes and distributes keys to other group users, who can then update the shared data. However, data modification requires the agreement of t or more out of t group users for the changes to be applied.

The communication layer forwards the sensed data by the end devices via the wired/wireless communication infrastructure. For example, in VANET, roadside units can collect traffic data from nearby vehicles with onboard units. To ensure the reliability of transmitted data, advanced communication technologies such as 5G and WLAN have emerged.

At the cognitive server layer, cloud storage nodes have unused capacity and offer paid storage services for user-transmitted outsourced data. Additionally, cloud servers can analyze and process real-time data. Cognitive IoT network managers can make adaptive decisions and optimize resources based on the analysis results provided by cognitive servers, enabling intelligence in IoT networks.

At the blockchain layer, all participants are in a P2P network, which is a distributed application architecture formed by a peer-to-peer computing model. All transactions generated in our protocol are recorded in the blockchain, which is transparent and tamper-resistant. Furthermore, smart contracts atop blockchain can audit whether the data is stored intact and provide fair compensation services.

3.2. System components

There are thirteen algorithms involved in the blockchain-based multiuser data integrity auditing protocol, namely, **Setup, KeyGen, Registration, Storage, DataProcess, KeyUpdate, Challenge, Response, Verification, DataUpdate, UserRevocation, Release and Deregistration**. The description of algorithms is as follows.

1. **Setup:** Executed by a trustworthy manner, which inputs security parameter λ. It generates system parameters PP to initial the protocol.
2. **KeyGen:** Executed by a data owner, which inputs a security parameter λ and the system parameter PP. It returns the owner's key pair and private keys for group users.
3. **Registration:** Executed by the node atop blockchain aiming to be a storage node. It returns a key pair for the storage node. Finally, the storage node deposits into the smart contracts.
4. **Storage:** Executed by a user intending to outsource data, this step involves making the deposit into the smart contracts and signing the transaction.
5. **DataProcess:** Executed by the data owner to preprocess files prior to outsourcing. This. The algorithm produces block tags for all data blocks and slices all data blocks into several parts. Outsource the data blocks to different storage nodes, and construct a rank-based Merkle tree.
6. **KeyUpdate:** Executed by the data owner. At each time slice, the data owner issues a private key update to group users, who then verify the validity of the update. If the update is valid, the group users apply the update to their private keys individually. Otherwise, they re-request the update from the data owner.
7. **Challenge:** Automatically triggered by smart contracts to generate a challenge set.
8. **Response:** Executed by the storage nodes storing the blocks in the challenge set, which returns the proof to demonstrate the integrity of challenged blocks.
9. **Verification:** Automatically executed by smart contracts for verifying the validity of proof from the storage node, which outputs *TRUE* for valid or $\perp$ for invalid proofs.

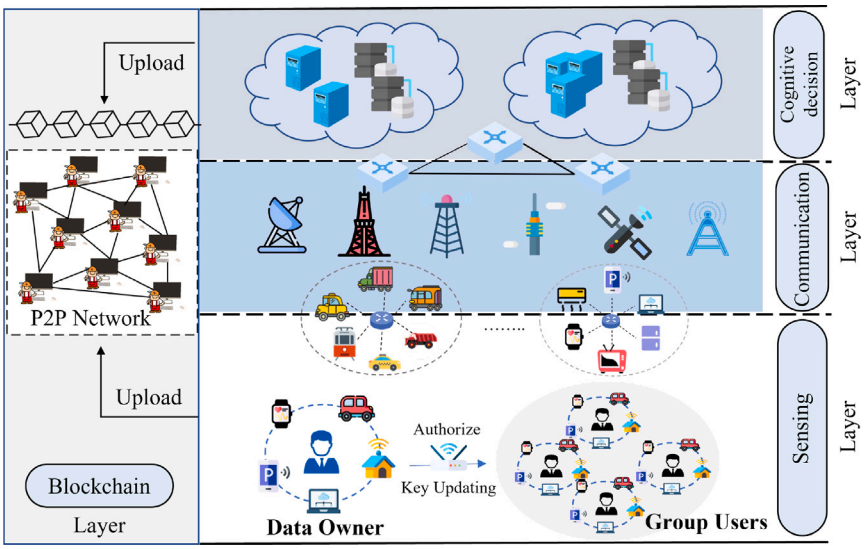


Fig. 1. The system framework of our protocol in cognitive IoT networks.

10. **DataUpdate:** Executed by group users intending to update the outsourced data. The update operation consists of data modification, insertion, or deletion. The update will be valid only when t or more of the K group users agree on the dynamic operation. The storage node handling the targeted block performs the requested update and provides the proof. Finally, group users verify the update proof and return *TRUE* if valid or $\perp$ otherwise.
11. **UserRevocation:** Executed by the data owner to revoke some malicious group users. During revocation, the data owner updates the public polynomial and adds the revoked user to a revocation list.
12. **Release:** Automatically handled by smart contracts under conditions such as a failed audit or data update ($\perp$), contract expiration, or when the data owner requests early retrieval of data before the contract's expiration.
13. **Deregistration:** Executed by a storage node intending to rescind from storage services before the contract expires or by the smart contract upon service contract expiration.

3.3. Security model

Here, we define the *completeness*, *soundness* and *fairness* for our proposal. First, we consider completeness in the following four cases.

- Any private key of the group user generated by an honest data owner by the **KeyGen** algorithm is able to pass the verification by group users with overwhelming probability.
- When interacting with an honest storage node, for all private keys sk_0 output by the **KeyGen** algorithm, for any file M, for all tags output by the **DataProcess** algorithm, it always returns *TRUE* by the **Verification** algorithm.
- After the data owner issues a private key update to group users, the updated key will pass the verification by group users with overwhelming probability.
- When the smart contract receives t or more than t requests of some update operation, it always outputs a valid update request to the storage nodes.

For soundness, we formalize a game involving a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$, specifically addressing potential collusion between storage nodes and the revoked group users. Let $\mathcal{L}_c$ be a list of group users that are corrupted by the adversary $\mathcal{A}$ in a time slice.

- **Setup:** $\mathcal{C}$ initializes the game by running the **Setup** algorithm to generate public parameters PP, then produces the private key sk_0 by the **KeyGen** algorithm. $\mathcal{C}$ shares PP with $\mathcal{A}$.
- **Query:** $\mathcal{C}$ responds these queries detailed below from $\mathcal{A}$ in polynomial time.
    1. $PrivateKey_Query$: The challenger $\mathcal{C}$ computes the group users' private keys, then sends $\{x_k\}_{k\in\mathcal{L}_c}$ and U_l to the adversary $\mathcal{A}$ in a time slice.
    2. Tag_Query: $\mathcal{A}$ selects a data block m_i at random and submits it to $\mathcal{C}$. Upon receiving m_i, $\mathcal{C}$ calculates its tag σ_i and sends σ_i back to $\mathcal{A}$.
    3. $DataUpdate_Query$: $\mathcal{A}$ selects a data block m_i for update and submits it to $\mathcal{C}$. $\mathcal{C}$ processes the update request, while $\mathcal{A}$ performs the associated update operation, outputs the update proof, and also submits it to $\mathcal{C}$. $\mathcal{C}$ then verifies the received proof and informs $\mathcal{A}$ of the result.
- **Challenge:** $\mathcal{C}$ issues a challenge set Q by the $Challenge$ algorithm to $\mathcal{A}$.
- **Forge:** $\mathcal{A}$ outputs a proof P_d.

Definition 1. A blockchain-based multiuser data integrity auditing protocol satisfies *soundness*, if there exists an *extractor* $E(\cdot)$, for any adversary $\mathcal{A}$, once $\mathcal{A}$ outputs a valid proof P_d for some file M, the extractor $E(\cdot)$ can retrieve the origin data M except with negligible probability, i.e., $1 - Pr[M \leftarrow E(PP, sk_0, P_d)] \leq negl(\cdot)$.

Fairness in this protocol is defined in terms of the equitable treatment of both the data owner and the storage nodes.

- The data owner retains full control over their data and can dynamically manage the group user. In the event of data corruption, the data owner is entitled to compensation, but only if there is verifiable proof of corruption. The audit process for the data owner should be publicly verifiable, preventing false claims or malicious accusations of data corruption.
- The storage nodes are compensated only if they meet the agreed-upon terms for data storage and integrity. If the node behaves maliciously, such as by failing to respond or submitting false proofs, they will face penalties.

4. Our construction

This section presents our design for a blockchain-based multiuser data integrity auditing protocol.

Setup. On input a security parameter λ. Let G_1, G_2, and G_T be groups of prime order p with a bilinear pairing. Let g_1, g_2, and $g_T = e(g_1, g_2)$ be generators of G_1, G_2, and G_T, respectively. Pick $\alpha \xleftarrow{R} Z_p$ as a secret value, and compute $g_1^{\alpha}, \ldots, g_1^{\alpha^N}, g_1^{\alpha^{N+2}}, \ldots, g_1^{\alpha^{2N}}, g_2^{\alpha}, \ldots, g_2^{\alpha^N}, g_T^{\alpha^{N+1}}$. Define four cryptographic hash function hash functions as follows: $H : \{0,1\}^* \to Z_p$, $H_1 : G_T \to \{0,1\}^{\lambda}$, $H_2 : \{0,1\}^* \to \{0,1\}^{\lambda}$ and $h : \{0,1\}^* \to \{0,1\}^{\lambda}$. Let $\mathcal{G}$ be a pseudorandom generator, which can generate a random number by affining from $\{\alpha, \ldots, \alpha^N\}$. Output the public parameter $PP = \{g_1, g_2, g_T = e(g_1, g_2), g_1^{\alpha}, \ldots, g_1^{\alpha^N}, g_1^{\alpha^{N+2}}, \ldots, g_1^{\alpha^{2N}}, g_2^{\alpha}, \ldots, g_2^{\alpha^N}, g_T^{\alpha^{N+1}}, H, H_1, H_2, h, \mathcal{G}\}$.

KeyGen. On input a security parameter λ and PP, the data owner U_0 randomly selects s_0, a as his private key using the pseudorandom generator $\mathcal{G}$, with its corresponding public key represented as ($V_1 = g_2^{s_0}, V_2 = g_2^{\alpha s_0}, y = g_1^{a}$). Assume that there are K users share the files, namely $W = \{U_1, \ldots, U_K\}$. Identities of these users, generated by $\mathcal{G}$, are represented as $\{ID_1, \ldots, ID_K\}$, and U_0 computes the polynomial $F(x) = \prod_{k=1}^{K}(x - H_2(ID_k))$. U_0 generates $t-1$ elements $a_1, a_2, \ldots, a_{t-1}$ by $\mathcal{G}$, and constructs the polynomial $f(x) = s_0 + a_1 x + a_2 x^2 + \cdots + a_{t-1}x^{t-1}$. U_0 computes private keys $x_k = f(ID_k)_{(1 \le k \le K)}$ for each group user, and computes $u_0 = g_1^{s_0}, u_l = g_1^{a_l}$, where $1 \le l \le t-1$. Then U_0 distributes $\{x_k\}_{1 \le k \le K}$ as private keys to the group users, and makes the $\{u_l\}_{1 \le l \le t-1}$ public. When each group user U_k receives the private key x_k, check whether

$$g_1^{x_k} \stackrel{?}{=} \prod_{l=0}^{t-1} u_l^{ID_k^l}. \tag{1}$$

If not, U_k re-requests his private key from U_0. Otherwise, x_k is valid.

Registration. A peer in the P2P network becomes a storage node SN by registering, and then the peer is able to invest its space for storage service. SN generates its public/private key pair (pk_{SN}, sk_{SN}), then deposits $(P+tf)$ *ETH* to the smart contract address, where P *ETH* is guarantee deposit for preventing its malicious behaviors and tf *ETH* is transaction fee for miners. Finally, SN signs the transactions using its private key.

Storage. To outsource a file for a duration T, the data owner U_0 deposits $(r + tf)$ *ETH* into the smart contract, where r *ETH* is rent fee of storage service and tf *ETH* is transaction fee for miners.

DataProcess. Before data outsourcing, U_0 performs the following steps to process the file.

1. TagGen: Compute a file identifier $h_1 = H(name)$ with the file name *name*, then divide the data file M into n blocks, each of which is further split into s sectors ($s < N$), i.e. $M = \{m_{ij}\}_{1 \le i \le n, 1 \le j \le s}$, where m_{ij} is the jth sector of the ith block. For each block m_i, U_0 computes a file tag σ_i as $\sigma_i = (g_1^{h_1} \cdot C_i)^{s_0} = (g_1^{h_1} \cdot \prod_{j=1}^{s} g_1^{m_{ij} \cdot \alpha^j})^{s_0}$ where $C_i = g_1^{\sum_{j=1}^{s} m_{ij}\alpha^j}$.
2. Outsourcing: The data owner U_0 randomly divides n data blocks into d parts and sends these blocks and corresponding tags to d different storage nodes $SN_1, \ldots, SN_d$. U_0 constructs a rank-based Merkle hash tree, and each storage node SN_i stores a subtree. The root nodes of subtrees are published to the blockchain, denoted as $\{h_{root_1}, \ldots, h_{root_d}\}$.

KeyUpdate. At each time slice, the data owner U_0 issues a key update for group users. τ is used to record the update times, $\tau = 1, 2, \ldots$. Pick a random $\beta \xleftarrow{R} Z_p$, and generate an one-way hash chain $\beta, H_1(\beta), H_1^2(\beta), \ldots, H_1^N(\beta)$. At the $\tau - th$ update, the data owner U_0 extracts the $H_1^{N-\tau}(\beta)$ from the hash chain to construct the polynomial $G(x) = H_1^{N-\tau}(\beta) \cdot \sum_{l=1}^{t-1} x^l$, and computes the update values as $Up_k^{(\tau)} = G_{\tau}(ID_k)$ $(1 \le k \le K)$. U_0 computes $U^{(\tau)} = g_1^{H_1^{N-\tau}(\beta)}$, and makes the $U^{(\tau)}$ public. When the group user U_k receives the update value, U_k computes $h_k = \sum_{l=1}^{t-1} ID_k^l$, and checks whether

$$g_1^{Up_k(\tau)} \stackrel{?}{=} (U^{(\tau)})^{h_k}. \tag{2}$$

If not, the group user U_k re-requests the update value from U_0. Otherwise, U_k applies the update as $x_k^{\tau} = x_k^{(\tau-1)} \cdot Up_k^{(\tau)}$.

Challenge. To check that outsourced data is stored intact, the smart contract automatically extracts six nonces in the most recent blocks on the blockchain and computes their average value, denoted as r_c. This value serves as the seed for pseudorandom permutation $f_{prf(r_c)}$. For $i \in [1, c]$, compute c challenge indexes by $f_{prf(r_c)}(i)$, and pick c random values $\theta_i \xleftarrow{R} Z_p$. c challenge indices form a sampling subset I, where $I \subseteq [n]$. The resulting challenge set $Q = \{(i, \theta_i)\}_{i \in I}$ is published to the blockchain for use in the verification process.

Response. Upon receiving a challenge set Q, the storage node SN_r checks whether there is an intersection between the storage index set R and the index set I in Q, i.e. $K = I \cap R$. If $K \neq \emptyset$, SN_r generates the response as follows. Firstly, SN_r computes $\mu_r = \sum_{i \in K} \sum_{j=1}^{s} \theta_i \gamma_j m_{ij}$, $\sigma_r = \prod_{i \in K} \sigma_i^{\theta_i}$, where $\gamma_j = H(Q, j)_{j \in [s]}$. Then SN_r aggregates $\pi_i = \prod_{j=1}^{s} \pi_{i,j}^{\gamma_j}$, $\pi_r = \Pi_{i \in K} \pi_i^{\theta_i}$ where $\pi_{i,j} = g_1^{\sum_{k=1, k \neq j}^{s} m_{ik} \alpha^{N+1-j+k}} = (C_i / g_1^{m_{ij}\alpha^j})^{\alpha^{N+1-j}}$ and $\pi_{i,j}$ can be pre-computed. Meanwhile, SN_r computes the auxiliary authentication information $\{\Omega_{i,r}\}_{i \in K}$ with the Merkle hash tree. The proof $Pf_r = \{\{\Omega_{i,r}\}_{i \in K}, \sigma_r, \mu_r, \pi_r, h_{root_r}\}$.

Verification. Run by the smart contract to handle multiple verification tasks from different storage nodes served for the same group. On input $L(|L| \le d)$ proofs $\{Pf_r\}_{r \in L}$ from the storage node $\{SN_r\}_{r \in L}$, the smart contract parses $\{Pf_r = \{\{\Omega_{i,r}\}_{i \in K_r}, \mu_r, \pi_r, h_{root_r}\}\}_{r \in L}$. For each storage node SN_r, the smart contract begins by verifying the auxiliary authentication information associated with the Merkel trees. Specifically, it computes a root node h'_{root_r} of these subtrees by using $\{\Omega_{i,r}\}_{i \in K_r, r \in L}$, then checks whether $h'_{root_r} \stackrel{?}{=} h_{root_r}$. If not, output $\perp$. Then, the smart contract aggregates L proofs and checks whether $e(\prod_{r \in L} \sigma_r, g_2^{\sum_{i=1}^{s} \alpha^{N+1-i}\gamma_i}) \stackrel{?}{=} e(\prod_{r \in L} \pi_r \cdot g_1^{h_1(\sum_{(i,\theta_i) \in Q} \sum_{j=1}^{s} \alpha^{N+1-j}\theta_i\gamma_j)}, V_1) \cdot e(g_1^{\alpha^N \sum_{r \in L} \mu_r}, V_2)$ where $\gamma_j = H(Q, j)_{j \in [s]}$ and $h_1 = H(name)$. In particular, $g_2^{\sum_{i=1}^{s} \alpha^{N+1-i}\gamma_i}$ and $g_1^{\sum_{(i,\theta_i) \in Q} \sum_{j=1}^{s} \alpha^{N+1-j}\theta_i\gamma_j}$ can be precomputed off-chain.If not, output $\perp$. Otherwise, output *TRUE*.

DataUpdate. Each group user is able to initiate updates to the outsourced data, while the update will be valid only when t or more of the K group users agree on it. In detail, each user publishes an update transaction with the transaction fee BT_{update}, which contains the data block to be updated along with the corresponding update operations: modification, insertion and deletion.

- Modification($\mathcal{M}$): Assume group users want to update the ith block m_i to m'_i.

  1. The group user U_k who agrees to the modification uses his own private key to compute $M_k = (\prod_{j=1}^{s} g_1^{(m'_{ij} - m_{ij})\alpha^j})^{x_k}$, and inputs M_k into the smart contract. Observed that U_k is allowed to partially modify m_i, for example $M_k = (g_1^{(m'_{ij} - m_{ij})\alpha^j})^{x_k}$ to update the jth sector m_{ij} in m'_{ij}. The smart contract begins by confirming the authenticity of the group user U_k, and checks whether $F(H_2(ID_k)) \stackrel{?}{=} 0$. If not, output $\perp$. Otherwise, after the smart contract has collected t valid update requests $\{M_k\}_{1 \le k \le t}$, it does the following computation: $\sigma_{i,\Delta} = \prod_{k=1}^{t}(M_k)^{\prod_{l=1, l \neq k}^{t} \frac{-ID_l}{ID_k - ID_l}}$. Then the smart contract outputs the update request $R_{update} = \{\mathcal{M}, i, m'_i, \sigma_{i,\Delta}\}$.
  2. If the index i in R_{update} is contained at the storage node SN_r, SN_r updates m_i by replacing it with m'_i and updates the corresponding tag σ_i to σ'_i, where $\sigma'_i = \sigma_i \cdot \sigma_{i,\Delta}$. Next, SN_r regenerates the leaf node $h(m'_i)$, reconstructs a new subtree and computes the corresponding root node h'_{root_r}. Finally, it outputs the update proof $P_{update} = \{h(m'_i), \Omega_{i,r}, h'_{root_r}\}$.

3. On receiving the update proof P_{update}, the group users reconstruct the subtree by utilizing $h(m_i')$ and $\Omega_{i,r}$, compute a root node $h^*_{root_r}$, and check if $h^*_{root_r} \stackrel{?}{=} h'_{root_r}$. If the check fails, it outputs $\perp$. Otherwise, it outputs *TRUE*.

- Insertion($\mathcal{I}$): Assume the group users want to insert a new block m_i' at the ith position.

1. A group user U_k who agrees to the insertion computes $I_k = (g_1^{H(name)} \cdot \prod_{j=1}^{s} g_1^{m'_{ij}\alpha^j})^{x_k}$ using his own private key and submits I_k to the smart contract. Then this contract begins by confirming the authenticity of U_k, and checks whether $F(H_2(ID_k)) \stackrel{?}{=} 0$. If not, output $\perp$. Otherwise, after the smart contract has collected t valid update requests $\{I_k\}_{1\le k\le t}$, it does the following computation: $\sigma_i' = \prod_{k=1}^{t}(I_k)^{\prod_{l=1,l\neq k}^{t}\frac{-ID_l}{ID_k-ID_l}}$. Then the smart contract outputs the update request $R_{update} = \{\mathcal{I}, i, m_i', \sigma_i'\}$.
2. If the index i in R_{update} is contained at the storage node SN_r, SN_r inserts m_i' with corresponding tag σ_i' at the ith position. SN_r re-generates the leaf node $h(m_i')$ for m_i', then constructs a new subtree and computes the corresponding root node h'_{root_r}. Output the update proof $P_{update} = \{h(m_i'), \Omega_{i,r}, h'_{root_r}\}$.
3. On receiving the update proof P_{update}, the group users reconstruct the subtree by utilizing $h(m_i')$ and $\Omega_{i,r}$, compute a root node $h^*_{root_r}$, and check if $h^*_{root_r} \stackrel{?}{=} h'_{root_r}$. If the check fails, it outputs $\perp$. Otherwise, it outputs *TRUE*.

- Deletion($\mathcal{D}$): Assume the group users want to delete block m_i at the ith position.

1. A group user U_k who agrees to the deletion inputs the position i and ID_k into the smart contract. Then this contract begins by confirming the authenticity of U_k, and checks whether $F(H_2(ID_k)) \stackrel{?}{=} 0$. If not, output $\perp$. Otherwise, after the smart contract has collected t valid update requests, it outputs the update request $R_{update} = \{\mathcal{D}, i, null, null\}$.
2. If the index i in R_{update} is contained at the storage node SN_r, SN_r deletes m_i and its tag σ_i. SN_r removes the leaf node $h(m_i)$ of m_i, reconstructs a new subtree and obtains the new root node h'_{root_r}. Output the update proof $P_{update} = \{h(m_{i-1}), \Omega_{i,r}, h'_{root_r}\}$.
3. On receiving the update proof P_{update}, the group users reconstruct the subtree by utilizing $h(m_{i-1})$ and $\Omega_{i,r}$, compute a root node $h^*_{root_r}$, and check if $h^*_{root_r} \stackrel{?}{=} h'_{root_r}$. If the check fails, it outputs $\perp$. Otherwise, it outputs *TRUE*.

UserRevocation. The data owner U_0 has the authority to revoke a group user U_k if U_k engages in any misbehavior. To do this, U_0 modifies the polynomial as $F(x)' = F(x)/(x - H_2(ID_k))$. The updated polynomial $F(x)'$ is then published as the new public polynomial. Additionally, U_0 adds U_k to the revocation list RL, which initially starts empty, as $RL \leftarrow RL \cup \{ID_k, x_k\}$.

Release. Once the following cases occur, a release transaction is published to the blockchain, accompanied by the transfer of the storage node's deposit from the smart contract.

1. **Verification** algorithm or **DataUpdate** algorithm output $\perp$. In this case, the deposit of the storage node *P ETH* is compensated to the data owner U_0 from the smart contract.
2. U_0 retrieves all data and terminates the storage service before it expires. In this case, *P ETH* is redeemed to the storage node from the smart contract.
3. The storage service has expired and the smart contracts redeem the deposit *P ETH* to the storage node.

Deregistration. When the storage node is not on lease, it can deregister the service by publishing a deregister transaction. Upon receiving the deregister transaction, If the node is confirmed to be unrented, the smart contract returns its deposit *P ETH* to it.

5. Security proof

This section provides a security proof for the protocol, together with an analysis of the fairness upheld among the participating parties.

5.1. Completeness

The following four key aspects demonstrate this completeness:

(1) The completeness of the private key for group users indicates that the private keys for any group user generated by an honest data owner satisfies the equation(Eq. (1)) as detailed below.

$$\prod_{\ell=0}^{t-1} u_\ell^{ID_k^\ell} = \prod_{\ell=0}^{t-1} g_1^{a_\ell ID_k^\ell} = g_1^{\sum_{\ell=0}^{t-1} a_\ell ID_k^\ell} = g_1^{f(ID_k)} = g_1^{x_k}$$

(2) The completeness of the proof of storage indicates that an honest storage node can always pass the **Verification** algorithm detailed as follows.

$$\begin{aligned}
&e(\prod_{r\in L}\pi_r \cdot g_1^{h_1(\sum_{(i,\theta_i)\in Q}\sum_{j=1}^{s}\alpha^{N+1-j}\theta_i\gamma_j)}, V_1)\cdot e(g_1^{\alpha^N\sum_{r\in L}\mu_r}, V_2)\\
&= e(\prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s}\pi_{i,j}^{\theta_i\gamma_j}\cdot\prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s} g_1^{h_1(\alpha^{N+1-j}\theta_i\gamma_j)}, V_1)\\
&\quad\cdot e(g_1^{\alpha^N\sum_{(i,\theta_i)\in Q}\sum_{j=1}^{s}\theta_i\gamma_j m_{ij}}, V_1{}^{\alpha})\\
&= e(\prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s}(\pi_{i,j}^{\theta_i\gamma_j}\cdot g_1^{h_1(\alpha^{N+1-j}\theta_i\gamma_j)}), V_1)\cdot e(\prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s} g_1^{\alpha^N\theta_i\gamma_j m_{ij}}, V_1{}^{\alpha})\\
&= \prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s}\left(e((C_i/g_1^{m_{ij}\alpha^j})^{\alpha^{N+1-j}}\cdot g_1^{h_1\alpha^{N+1-j}}, V_1{}^{\theta_i\gamma_i})\cdot e(g_1^{\alpha^{N+1}m_{ij}}, V_1{}^{\theta_i\gamma_j})\right)\\
&= \prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s}\left(e(C_i^{\alpha^{N+1-j}}\cdot g_1^{-m_{ij}\alpha^{N+1}}\cdot g_1^{m_{ij}\alpha^{N+1}}\cdot g_1^{h_1\alpha^{N+1-j}}, V_1{}^{\theta_i\gamma_j})\right)\\
&= \prod_{(i,\theta_i)\in Q}\prod_{j=1}^{s}\left(e((C_i\cdot g_1^{h_1})^{\theta_i s_0}, g_2{}^{\alpha^{N+1-j}\gamma_j})\right) = e(\prod_{r\in L}\sigma_r, g_2^{\sum_{j=1}^{s}\alpha^{N+1-i}\gamma_j})
\end{aligned}$$

(3) The completeness for updating the private key of group user indicates that the updated key can always satisfy the equation (Eq. (2)). This process applies to the τth update, where $\tau = 1, 2, \ldots$, as described below.

$$(U^{(\tau)})^{h_k} = (g_1^{H_1^{N-\tau(\beta)}})^{\sum_{\ell=1}^{t} ID_R^\ell} = g_1^{G(ID_k)} = g_1^{Up_k(\tau)}$$

Then, the polynomial $f_\tau(x)$ is:

$$\begin{aligned}
&f_\tau(x) = f_{\tau-1}(x) + G_\tau(x)\\
&= f_{\tau-2}(x) + G_{\tau-1}(x) + G_\tau(x) = \cdots\\
&= f_0(x) + \sum_{i=1}^{\tau} G_i(x)\\
&= \sum_{\ell=0}^{t-1} a_\ell x^\ell + \sum_{l=1}^{t-1}(\sum_{i=1}^{\tau} H^{N-i}(\beta)x^\ell)\\
&= s_0 + \sum_{\ell=1}^{t-1} a_\ell x^\ell + \sum_{l=1}^{t-1}(\sum_{i=1}^{\tau} H^{N-i}(\beta)x^\ell)\\
&= s_0 + \sum_{\ell=1}^{t-1} a_\ell{}^{(\tau)} x^\ell
\end{aligned}$$

where $a_\ell{}^{(\tau)} = a_\ell + \sum_{i=1}^{\tau} H^{N-i}(\beta)(1 \le \ell \le t-1)$, and the updated key of the group user U_k satisfies $x_k^\tau = f_\tau(ID_k)$.

(4) The completeness of the generation of data updating requests indicates that the valid request can always update the data correctly, which details the data modification as an example.

$$\sigma_i \cdot \sigma_{i,\Delta} = \sigma_i \cdot \prod_{k=1}^{t} (M_k)^{\prod_{l=1,l\neq k}^{t} \frac{-ID_l}{ID_k - ID_l}}$$

$$= \sigma_i \cdot \prod_{k=1}^{t} ((\prod_{j=1}^{s} g_1^{(m'_{ij}-m_{ij})\alpha^j})^{x_k \prod_{l=1,l\neq k}^{t} \frac{-ID_l}{ID_k-ID_l}})$$

$$= \sigma_i \cdot (\prod_{j=1}^{s} g_1^{(m'_{ij}-m_{ij})\alpha^j})^{\sum_{k=1}^{t} x_k \cdot \prod_{l=1,l\neq k}^{t} \frac{-ID_l}{ID_k-ID_l}}$$

$$= \sigma_i \cdot (\prod_{j=1}^{s} g_1^{(m'_{ij}-m_{ij})\alpha^j})^{s_0}$$

$$= (g_1^{h_1} \cdot \prod_{j=1}^{s} g_1^{m_{ij}\alpha^j})^{s_o} \cdot (\prod_{j=1}^{s} g_1^{(m'_{ij}-m_{ij})\alpha^j})^{s_0}$$

$$= (g_1^{h_1} \cdot \prod_{j=1}^{s} g_1^{m'_{ij}\alpha^j})^{s_o} = \sigma_i'$$

5.2. Soundness

Lemma 1 (*Collusion Resistance*). *If the hash function H_1 is cryptographic hash, and (t, n)-Shamir secret sharing scheme is secure, our protocol is collusion-resistance. That is, when the malicious server S colludes with less than t revoked users in a time slice $\tau(\tau = 1, 2, \ldots)$, it cannot extract any information about the private key of the data owner U_0 or private keys of other group users, nor can it generate a valid modification.*

Proof. Suppose $f_\tau(x)$ is the interpolation polynomial caused the success for the server S. The malicious server S successfully extracts the private key of U_0 or other group users when it colludes with $k(k < t)$ revoked users. This result indicates that it learned all the coefficients of $f_\tau(x)$. However, this is not feasible. If S could determine all the coefficients of $f_\tau(x)$ under these conditions, it would effectively break Shamir secret sharing scheme, which causes contradictions.

In fact, it is costly for a malicious server S to corrupt more than t revoked users in the time slice τ, which implies that the assumption is acceptable. When entering the next time slice $\tau + 1$, group users will update their private key. The private keys of k revoked users colluding with S in the time slice τ have expired in the time slice $\tau + 1$. Consequently, the server S needs to re-corrupt another group users, who is revoked in the time slice $\tau + 1$. Note that the threshold t is not updated in **KeyUpdate** algorithm. This result shows that in any time slice τ', S cannot successfully extract knowledge for the private key of U_0 or other group users for the same reason as in the time slice τ. As for the interpolation polynomial $f_{\tau+1}(x)$ in time slice $\tau + 1$, it is only related to $H_1^{N-(\tau+1)}(\beta)$ for updating. The server S cannot compute $H_1^{N-(\tau+1)}(\beta)$ after obtaining $H_1^{N-\tau}(\beta)$. Otherwise, the security of the one-way hash chain is broken by using S, which would subsequently break the security of H_1.

Moreover, if S attempts to generate a valid modification together with k revoked users, the smart contract is tasked with checking whether the private keys belonging to these users are valid. Nevertheless, these private keys of the revocation users have been removed from the polynomial $F(x)$, and their identities have been recorded in the revocation list RL, so the modification is not feasible. Therefore, collusion resistance holds. □

Theorem 1. *If the l-wBDHE* assumption holds and the BLS signature scheme [45] is unforgeable. Then, in the algebraic group model and random oracle model, our protocol satisfies the soundness defined in Section 3.3. Specifically, the probability of a forged a proof being accepted by the **Verification** algorithm is negligible, even if the adversary colludes with revocation users.*

Proof. We prove this theorem with the following games:

Game 0. Corresponds to the soundness game described in Section 3.3, where the interaction takes place between a challenger C and an adversary $\mathcal{A}$.

Game 1. Corresponds to Game 0, except that C maintains a list to track the private keys of group users issued during the private key query phase. Note that the number of queries within a time slice is less than t, and the queries are repeated in polynomial time slices. If the adversary $\mathcal{A}$ generates a valid group user private key, including the private key of U_0, but does not appear in the private key list by C, then C aborts and declares failure.

Analysis. If $\mathcal{A}$ succeeds in causing the failure of Game 1, then $\mathcal{A}$ can be leveraged to contradict Lemma 1. Specifically, if $\mathcal{A}$ produces a valid private key that was not issued by the challenger C during the private key query phase, this implies that $\mathcal{A}$ has either reconstructed the private key of U_0 or generated a private key of another group user without querying it—both of which break the security of Lemma 1. Conversely, if $\mathcal{A}$ cannot cause the failure of Game 1, its view remains indistinguishable from that in Game 0. Here, the advantage of adversary $\mathcal{A}$ in Game 1 is denoted as $\mathbf{Adv}_{\mathcal{A},\Pi}^{Game_1}(\lambda) = \epsilon_1 \leq \mathbf{Adv}_{\mathcal{A},Shamir}(\lambda) + \mathbf{Adv}_{\mathcal{A},H_1}(\lambda)$, where $\mathbf{Adv}_{\mathcal{A},Shamir}(\lambda)$ represents the adversary's advantage in breaking the security of the Shamir secret sharing scheme, and $\mathbf{Adv}_{\mathcal{A},H_1}(\lambda)$ represents the adversary's advantage in breaking the cryptographic hash function H_1. Thus, $\mathcal{A}$ cannot succeed in generating a valid, non-inquired private key within a time slice τ without contradicting Lemma 1.

Game 2. Corresponds to Game 1, with the following modification: C keeps the list that records all signed tags associated with data blocks that were generated during the tag query phase. If $\mathcal{A}$ generates a valid tag not recorded in this list, C aborts.

Analysis. Suppose $\mathcal{A}$ outputs a valid and non-inquired tag $\sigma_i{}^*$, which causes the failure of Game 1. It is noted that is the same as Game 1, except that the original tag generated by the group users is replaced with the valid forged tag $\sigma_i{}^*$. According to Lemma 1, $\mathcal{A}$ cannot extract the private key of U_0 even when colluding with t revoked users. Therefore, $\mathcal{A}$ must have produced a valid BLS signature without access to the data owner's private key. If $\mathcal{A}$ succeeds in Game 2 with non-negligible probability, then the adversary's advantage is denoted as $\mathbf{Adv}_{\mathcal{A},\Pi}^{Game_2}(\lambda) = \epsilon_2 \leq \mathbf{Adv}_{\mathcal{A},\mathcal{BLS}}^{\text{EU-CMA}}(\lambda)$, where $\mathbf{Adv}_{\mathcal{A},\mathcal{BLS}}^{\text{EU-CMA}}(\lambda)$ denotes the advantage of $\mathcal{A}$ in breaking the unforgeability of the BLS signature scheme. This implies that a successful forgery in Game 2 would lead to a violation of the unforgeability of the BLS signature.

Game 3. Corresponds to Game 2 with the following modification: C keeps the list that records all responses to the block tag queries issued by $\mathcal{A}$. If $\mathcal{A}$ successfully outputs an acceptable proof, but the aggregated tag for the challenged data blocks $\sigma^* \neq \Pi_{(i,\theta_i)\in Q} \sigma_i^{\theta_i}$, where Q is the challenge set generated by C, then C aborts.

Analysis. Suppose the data blocks causing the failure of Game 3 are $M = \{m_{ij}\}_{1\leq i\leq n, 1\leq j\leq s}$, with corresponding tags $\{\sigma_1, \ldots, \sigma_n\}$. For the challenge set $Q = \{i, \theta_i\}_{i\in I}$, $\mathcal{A}$ outputs a response $Pf^* = \{\sigma^*, \mu^*, \pi^*\}$, resulting in failure. Let $Pf = \{\sigma, \mu, \pi\}$ be the expected response corresponding to Q and $\gamma_j = H(Q, j)_{1\leq j\leq s}$, where $\sigma = \Pi_{(i,\theta_i)\in Q} \sigma_i^{\theta_i}, \pi = \prod_{(i,\theta_i)\in Q} \prod_{j=1}^{s} \pi_{i,j}^{\theta_i\gamma_j}$ and $\mu = \sum_{(i,\theta_i)\in Q} \sum_{j=1}^{s} \theta_i\gamma_j m_{ij}$. According to completeness, an expected response follows, i.e., $e(\sigma, g_2^{\sum_{j=1}^{s} \alpha^{N+1-i}\gamma_j}) = e(\pi \cdot g_1^{h_1(\sum_{(i,\theta_i)\in Q} \sum_{j=1}^{s} \alpha^{N+1-j}\theta_i\gamma_j)}, V_1) \cdot e(g_1^{\alpha^N \mu}, V_2)$. In consideration of the failure, the equation holds for the response of $\mathcal{A}$, i.e., $e(\sigma^*, g_2^{\sum_{j=1}^{s} \alpha^{N+1-i}\gamma_j}) = e(\pi^* \cdot g_1^{h_1(\sum_{(i,\theta_i)\in Q} \sum_{j=1}^{s} \alpha^{N+1-j}\theta_i\gamma_j)}, V_1) \cdot e(g_1^{\alpha^N \mu^*}, V_2)$, but $\sigma^* \neq \sigma$. If $\pi = \pi^*$ and $\mu = \mu^*$, then $\sigma^* = \sigma$ can be inferred, which is contradictory. Therefore, let $\pi/\pi^* \neq 1$ or $\Delta \equiv_p \mu - \mu^* \equiv_p \sum_{(i,\theta_i)\in Q} \sum_{j=1}^{s} m_{ij}\theta_i\gamma_j - m_{ij}^*\theta_i\gamma_j \not\equiv_p 0$, where (π^*, μ^*) is simulated by $M^* = \{m_{ij}{}^*\}_{1\leq i\leq n, 1\leq j\leq s}$, PP and $M \neq M^*$ in algebraic group model.

Here we show the case where $\Delta \equiv_p 0$ and $\pi/\pi^* \neq 1$. We declare that this happens in a lucky challenge for the adversary when $M \neq M^*$, but $\sum_{(i,\theta_i)\in Q} \sum_{j=1}^{s} m_{ij}\theta_i\gamma_j - m_{ij}^*\theta_i\gamma_j \equiv_p 0$. Note that $\{\theta_i\}_{1\leq i\leq n}, \{\gamma_j\}_{1\leq j\leq s}$ are random numbers generated by the simulator B_3, which implies that $\Delta \equiv_p$

0 is equivalent to the linear combination of $\{\theta_i\}_{1\le i\le n}, \{\gamma_j\}_{1\le j\le s}$ being equal to 0. Consequently, the probability of this challenge occurring is at most $1/p$, which is negligible.

Then we show the case where $\Delta \not\equiv_p 0$. We prove that we can break the l-wBDHE* assumption by using the simulator $\mathcal{B}$ if $\mathcal{A}$ can win Game 3 in this case. Note that the parameters V_1, V_2 are output by $\mathcal{B}$ though selecting $s_0 \overset{R}{\leftarrow} Z_p$ and computing $V_1 = g_2^{s_0}, V_2 = g_2^{\alpha s_0}$, as well as the responses to the tag queries generated though s_0. We derive the following equation by dividing the expression for σ by that for σ^* in the **Verification** algorithm,i.e.,

$$e(\sigma/\sigma^*, g_2^{\sum_{j=1}^{s} \alpha^{N+1-i}\gamma_j}) = e(\pi/\pi^*, V_1)\cdot e(g_1^{\alpha^N\Delta}, V_2)$$
$$= e(\pi/\pi^*, g_2^{s_0})\cdot e(g_1^{\alpha^N\Delta}, g_2^{\alpha s_0})$$
$$= e((\pi/\pi^*)^{s_0}, g_2)\cdot e(g_1^{s_0\alpha^N\Delta}, g_2^{\alpha})$$

Then we have:

$$e((\sigma/\sigma^*)^{\sum_{j=1}^{s}\alpha^{N+1-i}\gamma_j}\cdot(\pi/\pi^*)^{-s_0}, g_2) = e(g_1^{s_0\alpha^N\Delta}, g_2^{\alpha})$$

Observed that the terms of σ, π in the left equation is simulated by $\mathcal{B}$ through the parameters $\{g_1^{\alpha}, \ldots, g_1^{\alpha^N}, g_1^{\alpha^{N+2}}, \ldots, g_1^{\alpha^{2N}}\}$, and σ^*, π^* are output by $\mathcal{A}$. Thus, we can extract $g_1^{\alpha^{N+1}} = \left((\sigma/\sigma^*)^{\sum_{j=1}^{s}\alpha^{N+1-i}\gamma_j}\cdot(\pi/\pi^*)^{-s_0}\right)^{-(s_0\Delta)}$ except that $\Delta \equiv_p 0$. Nevertheless, we prove that the probability of $\Delta \equiv_p 0$ is $1/p$. Thus, the solution is to break l-wBDHE* assumption whether $\pi/\pi^* = 1$ or not.

Then, the distinguishing advantage of adversary $\mathcal{A}$ between Game 2 and Game 3 is given by $\mathbf{Adv}_{\mathcal{A},\Pi}^{Game_3}(\lambda) = \epsilon_3 \le \frac{1}{p} + \left(1 - \frac{1}{p}\right)\cdot\mathbf{Adv}_{\mathcal{A}}^{l\text{-wBDHE}^*}(\lambda)$, where $\mathbf{Adv}_{\mathcal{A}}^{l\text{-wBDHE}^*}(\lambda)$ denotes the advantage of $\mathcal{A}$ in solving the l-wBDHE* problem. Therefore, if ϵ_3 is non-negligible, then one can construct a reduction that uses $\mathcal{A}$'s success to solve the l-wBDHE* problem with non-negligible probability, contradicting the assumed hardness of this problem.

Game 4. Corresponds to Game 2 with the following modification: If $\mathcal{A}$ submits an acceptable proof $Pf^* = \{\sigma^*, \mu^*, \pi^*\}$, but $\{\mu^*, \pi^*\}$ consists of M^* that are different from the challenged data blocks, then C aborts.

Analysis. Suppose that the data blocks causing the failure of Game 3 are $M = \{m_{ij}\}_{1\le i\le n, 1\le j\le s}$, with corresponding tags $\{\sigma_1, \ldots, \sigma_n\}$, the expected proof is $Pf = \{\sigma, \mu, \pi\}$ for $Q = \{i, \theta_i\}_{i\in I}$, and $\mathcal{A}$'s proof output be $Pf^* = \{\sigma^*, \mu^*, \pi^*\}$. Since $Pf^* = \{\sigma^*, \mu^*, \pi^*\}$ is acceptable, it indicates that $\{\sigma^*, \mu^*, \pi^*\}$ satisfies the verification equation, i.e., $e(\sigma^*, g_2^{\sum_{j=1}^{s}\alpha^{N+1-i}\gamma_j}) = e(\pi^*\cdot g_1^{h_1(\sum_{(i,\theta_i)\in Q}\sum_{j=1}^{s}\alpha^{N+1-j}\theta_i\gamma_j)}, V_1)\cdot e(g_1^{\alpha^N\mu^*}, V_2)$. In Game 3, we proved that the probability of $\sigma^* = \sigma$ is overwhelming, so if $\pi^* = \pi$, then $\mu^* = \mu$. However, we show that a lucky challenge occurring is negligible when μ^* consists of M^* and $M^* \ne M$ in Game 3. If $\pi^* \ne \pi$ and $\mu^* = \mu$, it leads to the contradictory by completeness. Thus, let $\Delta \equiv_p \mu - \mu^* \equiv_p \sum_{(i,\theta_i)\in Q}\sum_{j=1}^{s} m_{ij}\theta_i\gamma_j - m_{ij}^*\theta_i\gamma_j$ and $\Delta \not\equiv_p 0$.

Here we show that if $\mathcal{A}$'s success in Game 4 is non-negligible, then the simulator $\mathcal{B}$ can be constructed to break the l-wBDHE* assumption by using $\mathcal{A}$. Let $h_M^Q = g_1^{h_1(\sum_{(i,\theta_i)\in Q}\sum_{j=1}^{s}\alpha^{N+1-j}\theta_i\gamma_j)}$,We show the following equations when $\sigma^* = \sigma$ and $e(\sigma, g_2^{\sum_{j=1}^{s}\alpha^{N+1-i}\gamma_j}) = e(\sigma^*, g_2^{\sum_{j=1}^{s}\alpha^{N+1-i}\gamma_j})$.

$$e(\pi\cdot h_M^Q, V_1)\cdot e(g_1^{\alpha^N\mu}, V_2) = e(\pi^*\cdot h_M^Q, V_1)\cdot e(g_1^{\alpha^N\mu^*}, V_2)$$

Rearranging terms yields:

$$e(\pi\cdot(\pi^*)^{-1}, V_1) = e(g_1^{\alpha^N(-\Delta)}, V_1^{\alpha})$$

Note that the terms of $\{\pi, \mu\}$ are simulated by $\mathcal{B}$ though the parameters $\{g_1^{\alpha}, \ldots, g_1^{\alpha^N}, g_1^{\alpha^{N+2}}, \ldots, g_1^{\alpha^{2N}}\}$, $\{\pi^*, \mu^*\}$ is output by the adversary, and $\Delta \not\equiv_p 0$. We can extract $g_1^{\alpha^{N+1}} = \left(\pi\cdot(\pi^*)^{-1}\right)^{\Delta}$, which is the solution of l-wBDHE* problem.

Then, the distinguishing advantage of adversary $\mathcal{A}$ between Game 3 and Game 4 is defined as $\mathbf{Adv}_{\mathcal{A},\Pi}^{Game_4}(\lambda) = \epsilon_4 \le \mathbf{Adv}_{\mathcal{A}}^{l\text{-wBDHE}^*}(\lambda)$, Therefore, if ϵ_4 is non-negligible, a simulator $\mathcal{B}$ can be constructed that uses $\mathcal{A}$'s ability to distinguish between Game 3 and Game 4 to solve the l-wBDHE* problem with non-negligible probability.

Wrapping up. In Game 4, the adversary, even when colluding with revoked users, is restricted to output an acceptable forged proof. We claim that if the l-wBDHE* assumption holds and BLS signature is unforgeable, the success advantage of an algebraic adversary in Game 4 compared to Game 0, denoted $\mathbf{Adv}_{\mathcal{A},\Pi}^{Game_4}(\lambda) - \mathbf{Adv}_{\mathcal{A},\Pi}^{Game_1}(\lambda) = \epsilon_1 + \epsilon_2 + \epsilon_3 + \epsilon_4$, is negligible. □

5.3. Fairness

Outsourcing storage is a commercial activity that benefits both the data owner and the storage provider. The data owner pays rent fees to receive storage services, while the storage provider earns rewards for offering these services. However, if either party acts maliciously, it can cause harm to the other, highlighting the importance of fairness in the protocol.

To ensure fairness, we have designed a system based on smart contracts. These smart contracts enforce that both parties interact in good faith by implementing a time-locked deposit mechanism, which ensures that both sides fulfill their obligations within a defined time frame. Additionally, challenge set generation and proof verification are managed by the smart contract, rather than relying on any single party, preventing manipulation by either party.

If the server behaves maliciously by failing to respond or providing a forged proof, the deposit will be forfeited and compensated to the data owner. On the other hand, if the server is honest, it can redeem the deposit and receive the corresponding commission from the user. Furthermore, the protocol ensures that the data owner retains control over the shared data at all times. The data owner's control is also extended to the group's private key, which is issued and updated by the data owner, ensuring that only authorized parties can access or verify the data.

6. Implementation

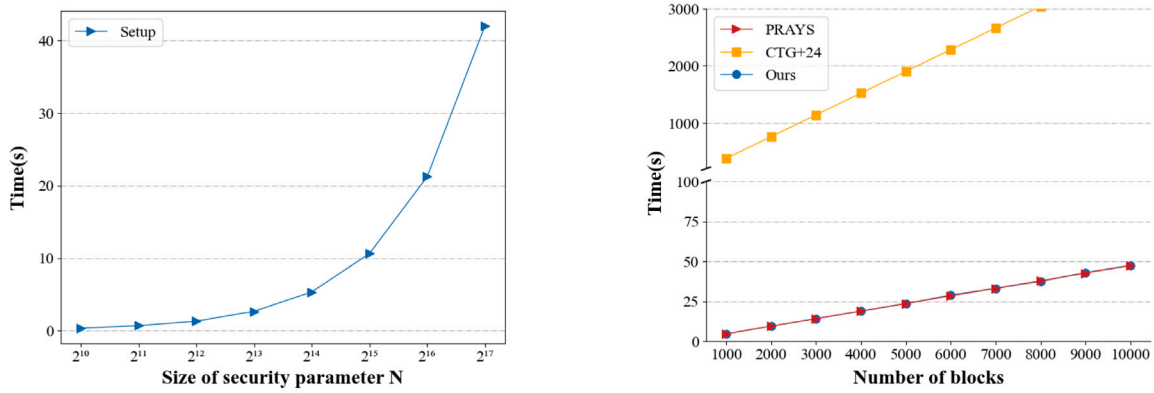
To evaluate the proposed protocol and demonstrate its practicality, we developed a prototype system. The evaluation is divided into two parts: the on-chain component, which measures gas consumption for blockchain-based operations, and the off-chain component, which measures time overhead for off-chain computations. In order to further illustrate the practicality of the proposed solution, we replicated PRAYS [29] and CTG+24 [33] under the same experimental configuration, and conducted a comparative analysis of the execution times to establish a general performance benchmark.

6.1. Off-chain implementation

All experiments have been implemented on a desktop using an Ubuntu 16.04LTS with Intel i5-7200U CPU @ 2.50 GHz and 4 GB RAM. The implementation was performed in C++ using the ate library,[2] the libff library,[3] and the libfqfft library,[4] which provide finite field and elliptic curve operations, optimized Barreto–Naehrig(BN) curve pairing operations, and fast Fourier transform in finite field, respectively. In detail, the shared file size is 156 MB split into 10 000 blocks. Each block consists of 256 sectors and each data size in a sector is 64 Bytes. In addition, our experiments simulate eight storage nodes, and we set the group user threshold to 20. All experimental results are averaged over 10 trials, which are presented in Figs. 2, 3 and 4.

First, the time consumption of the **Setup** algorithm is evaluated, as its performance depends solely on the security parameter N. We vary N from 2^{10} to 2^{17}, increasing it twice each time. Fig. 2(a) shows the consumption of the **Setup** algorithm increases linearly with N. When

[2] https://github.com/herumi/ate-pairing.
[3] https://github.com/scipr-lab/libff.
[4] https://github.com/scipr-lab/libfqfft.



(a) **Setup** algorithm for different sizes of security parameter

(b) **DataProcess** algorithm for various number of blocks

Fig. 2. Time consumption of **Setup** and **DataProcess** algorithms.

$N = 2^{17}$, the time consumption of this algorithm is 42.01 s, which is acceptable since the algorithm is run only once in the system and is used repeatedly by all participants.

Second, we evaluate the **DataProcess** algorithm under varying block counts from 1000 to 10,000 (in steps of 1000), corresponding to file sizes ranging from 15.6MB to 156MB. Each data block consists of 256 sectors of 64 Bytes, and all experiments on PRAYS [29], CTG+24 [33], and our protocol rely on multi-threaded modules that perform heavy yet independent modular exponentiation in parallel, with computation split among eight workers for approximately 5x speedup over sequential execution.[5] As shown in Fig. 2(b), our method exhibits near-linear time complexity while significantly outperforming PRAYS [29] and CTG+24 [33]. Compared to PRAYS [29], our approach demonstrates comparable efficiency at small scales but gains marginal advantages as block counts increase (e.g., 47.52 s vs. 47.32 s for 10,000 blocks). Meanwhile, against CTG+24 [33], which uses a proxy signature scheme without dividing data into smaller sectors – treating each domain element as an entire data block – we achieve approximately 80 × faster execution. By grouping multiple domain elements within a single block, our protocol and PRAYS [29] prevent an explosion in the total number of data blocks, thus substantially reducing the computational overhead. Notably, although the **DataProcess** step is computationally intensive, it is only performed once per file and thus remains practical for real-world deployments.

Moreover, we evaluate the cost during the auditing phase. With a fixed 156MB file divided into 10,000 data blocks, we evaluate the **Response** algorithm and **Verification** algorithm separately as the challenge blocks increases from 100 to 1000 in steps. Note that the cost of the off-chain **Verification** algorithm refers to pre-computation. As shown in Fig. 3(a), when the challenge size reaches 1000 blocks, our method requires only 2.81 ms for a single storage node to generate the **Response**, whereas PRAYS [29] and CTG+24 [33] take 2.97 ms and 2.95 ms, respectively. For a challenge size of 1000 blocks and responses aggregated from eight storage nodes, the off-chain verification with pre-computation in our scheme completes in just 94.58 ms, compared to 1003.6 ms under PRAYS [29] and 188 017.5 ms under CTG+24 [33]. These results underscore our significantly lower overhead in both response generation and verification, demonstrating that off-chain verification with pre-computation is cost-effective and reduces on-chain gas consumption.

[5] This configuration reflects the upper limit of physical threads in our environment. The computation uses independent modular exponen tiations, which map naturally to parallel processing. A single-producer, multiple-consumers pattern is adopted to distribute tasks, and intermediate results are collected via message passing.

Table 2
Gas cost of functions in *Audit* contract.

Functions	**Register**	**Storage**	**Challenge**	**Response**
Gas	47,315	53,812	287,938	95,817
Functions	**Verification**	**DataUpdate**	**Release**	**Deregistration**
Gas	639,723	368,493	67,392	55,653

Finally, we evaluate the consumption of group user operations. We test the consumption of **KeyGen** algorithm and **KeyUpdate** algorithm by varying the user group size from 10 to 100, with increments of 10. Fig. 4(a) shows that the **KeyGen** and **KeyUpdate** algorithms exhibit linear growth with respect to the number of group users. Additionally, **KeyUpdate** algorithm is more efficient than the **KeyGen** algorithm. For instance, when the group size reaches 100 users, the time required to update all users' keys is just 18.24 ms. Additionally, we test the computational overhead of the **UserRevocation** algorithm for different numbers of users. We set 1000 group users and vary the number of revoked users from 10 to 100, increasing in steps of 10. Fig. 4(b) shows that the consumption of the **UserRevocation** algorithm increases proportionally with the number of revoked users. Notably, even when 100 users are revoked, the operation completes in just 1.21 ms, which is exciting.

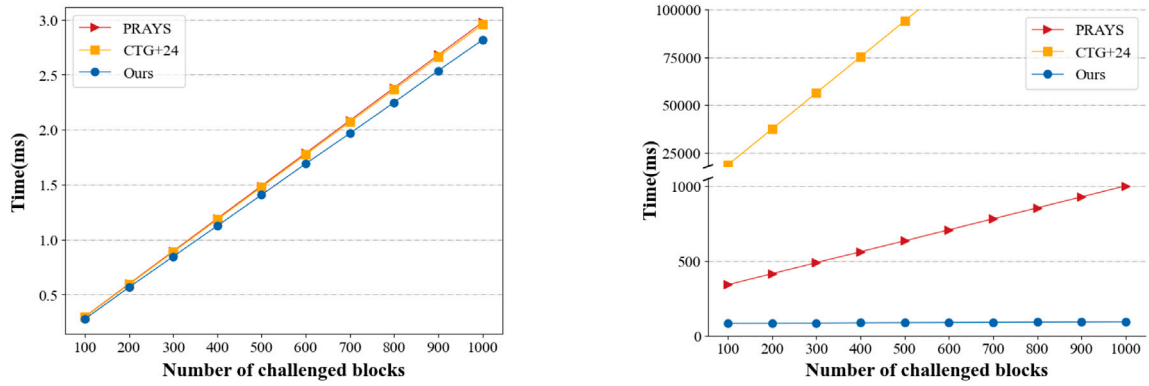
6.2. On-chain implementation

The on-chain operations are implemented on the private test chain of Ethereum via go-ethereum[6] and truffle,[7] where truffle is one of the most comprehensive suite tools for smart contract development. We implemented a *Audit* contract by using the elliptic-curve-solidity library,[8] which is an implementation of elliptic curve arithmetical operations written in Solidity.

Our *Audit* contract is triggered by calling the specific functions, including **Register**, **Storage**, **Challenge**, **Response**, **Verification**, **DataUpdate**, **Release**, **Deregistration**. We set up the experiment using a file size of 15.6MB, which was divided into 1000 equal-sized blocks and distributed across 8 distinct storage nodes. During each audit phase, 20 blocks were randomly selected as the challenge set to verify data integrity. The corresponding gas costs for each contract function under this setting are shown in Table 2.

Firstly, the storage node declares its storage service on the blockchain with a call to the **Register** function by paying the deposit,

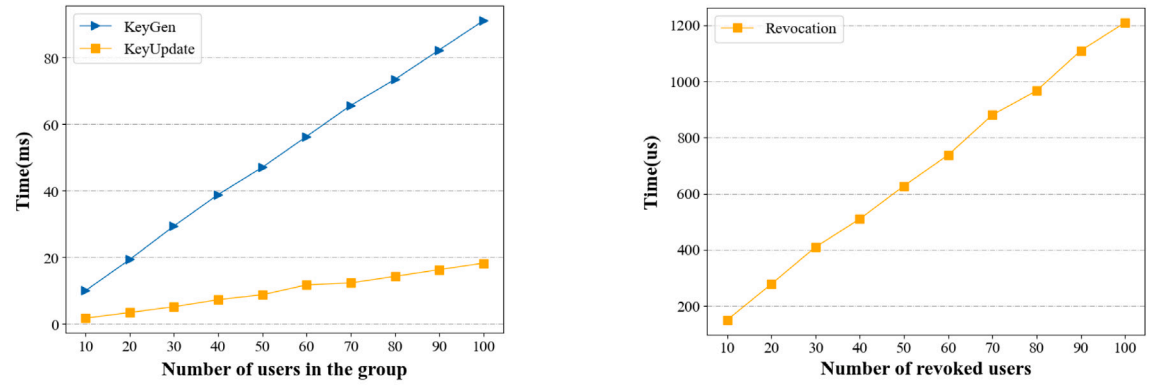
[6] https://github.com/ethereum/go-ethereum/.
[7] https://trufflesuite.com/.
[8] https://github.com/witnet/elliptic-curve-solidity.



(a) **Response** algorithm for various number of challenged blocks

(b) **Verification** algorithm for various number of challenged blocks

Fig. 3. Time consumption for the auditing phase.



(a) **KeyGen** and **KeyUpdate** algorithms for various number of users

(b) **UserRevocation** algorithm for various number of revoked users

Fig. 4. Time consumption of group user operations.

which requires 47,315 gas. To establish a lease with a storage node, a user invokes the **Storage** function with outsourced file size, threshold, and storage duration, which costs 53,812 gas. Before the storage expires, the user can call the **Challenge** function for checking that the cloud files are stored intact. We fix the challenge size to 20 and invoke the **Challenge** function at a cost of 281,938 gas. Then the storage node uploads the storage proof to the *Audit* contract by calling the **Response** function, which costs 95,817 gas. After receiving the storage proofs from 8 storage nodes, the *Audit* contract is triggered to execute the **Verification** function to verify the validity of these proofs, which costs 639,723 gas. We set the threshold to 10 to evaluate the gas cost of **DataUpdate** function. Data insertion is considered expensive, costing 408,493 gas to update the data. The **Release** function is called to terminate the storage service, which costs 67,392 gas. Finally, the storage node logs out of the storage service by calling the **Deregistration** function, which costs 55,653 gas.

7. Conclusion

In this paper, we focused on integrity auditing of shared data for decentralized storage in cognitive IoT, and proposed a blockchain-based multiuser data integrity audit protocol. In our protocol, files stored in a decentralized manner support batch auditing for integrity and dynamic data updating by employing the proposed tags. In particular, during the audit phase, the storage nodes can efficiently respond based on the proposed homomorphic tags by pre-computation. Note that an efficient group user management is proposed to support dynamic groups, including secure and efficient group user revocation and verifiable group user key updates. Moreover, we formalized a security model to resist collusion attacks, and the security proof demonstrates the completeness and soundness of our protocol, especially against collusion attacks between multiple revoked users and storage nodes. Due to the deposit mechanism atop the blockchain, our protocol can guarantee fairness between users and storage nodes. Our performance evaluation showed that our protocol is efficient both on-chain and off-chain, demonstrating its practicality and suitability for lightweight devices in cognitive IoT.

CRediT authorship contribution statement

Min Xie: Writing – original draft, Methodology, Investigation. **Yong Yu:** Writing – review & editing, Methodology, Formal analysis, Funding acquisition. **Ruonan Chen:** Writing – review & editing, Methodology, Formal analysis. **Yanqi Zhao:** Writing – review & editing, Investigation, Formal analysis. **Jianting Ning:** Writing – review & editing, Methodology, Formal analysis. **Xiaoyi Yang:** Software, Investigation. **Zoe L. Jiang:** Writing – review & editing, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Cryptography Science Foundation (2025NCSF02025), National Natural Science Foundation of

China (U24B20149, 62272385, 72442029, U23A20302, 62311540-156), the Key Research and Development Program of Shaanxi (2024GX-ZDCYL-01-09, 2024GX-ZDCYL-01-15), Major Program of Shandong Provincial Natural Science Foundation for the Fundamental Research (ZR2022ZD03), The Youth Innovation Team of Shaanxi Universities (24JP180), and Shaanxi Distinguished Youth Project (2022JC-47).

Data availability

No data was used for the research described in the article.

References

[1] Q. Wu, G. Ding, Y. Xu, S. Feng, Z. Du, J. Wang, K. Long, Cognitive Internet of Things: A new paradigm beyond connection, IEEE Internet Things J. 1 (2) (2014) 129–143.
[2] A. Kaur, K. Kumar, A. Prakash, R. Tripathi, Imperfect CSI-based resource management in cognitive IoT networks: A deep recurrent reinforcement learning framework, IEEE Trans. Cogn. Commun. Netw. 9 (5) (2023) 1271–1281.
[3] Z. Li, B. Chang, S. Wang, A. Liu, F. Zeng, G. Luo, Dynamic compressive wide-band spectrum sensing based on channel energy reconstruction in cognitive internet of things, IEEE Trans. Ind. Inform. 14 (6) (2018) 2598–2607.
[4] W. Zheng, C. Lai, D. He, N. Kumar, B. Chen, Secure storage auditing with efficient key updates for cognitive industrial IoT environment, IEEE Trans. Ind. Inform. 17 (6) (2021) 4238–4247.
[5] J. Yu, B. Yan, H. Qi, S. Wang, W. Cheng, An efficient and secure data sharing scheme for edge-enabled IoT, IEEE Trans. Comput. 73 (1) (2024) 178–191.
[6] H. Xie, J. Zheng, T. He, S. Wei, C. Hu, TEBDS: A trusted execution environment-and-blockchain-supported IoT data sharing system, Future Gener. Comput. Syst. 140 (2023) 321–330.
[7] G. Ateniese, R.C. Burns, R. Curtmola, J. Herring, L. Kissner, Z.N.J. Peterson, D.X. Song, Provable data possession at untrusted stores, in: Proceedings of CCS, 2007, pp. 598–609.
[8] A. Juels, B.S. Kaliski Jr., Pors: proofs of retrievability for large files, in: Proceedings of CCS, 2007, pp. 584–597.
[9] H. Shacham, B. Waters, Compact proofs of retrievability, J. Cryptology 26 (3) (2013) 442–483.
[10] G. Ateniese, R.D. Pietro, L.V. Mancini, G. Tsudik, Scalable and efficient provable data possession, in: Proceedings of SECURECOMM, 2008, p. 9.
[11] C.C. Erway, A. Küpçü, C. Papamanthou, R. Tamassia, Dynamic provable data possession, in: E. Al-Shaer, S. Jha, A.D. Keromytis (Eds.), Proceedings of CCS, 2009, pp. 213–222.
[12] Q. Wang, C. Wang, J. Li, K. Ren, W. Lou, Enabling public verifiability and data dynamics for storage security in cloud computing, in: M. Backes, P. Ning (Eds.), Proceedings of ESORICS, Vol. 5789, 2009, pp. 355–370.
[13] C. Yang, Y. Liu, F. Zhao, S. Zhang, Provable data deletion from efficient data integrity auditing and insertion in cloud storage, Comput. Stand. Interfaces 82 (2022) 103629.
[14] Y. Ren, D. Huang, W. Wang, X. Yu, BSMD: A blockchain-based secure storage mechanism for big spatio-temporal data, Future Gener. Comput. Syst. 138 (2023) 328–338.
[15] Y. Ren, Z. Lv, N.N. Xiong, J. Wang, HCNCT: A cross-chain interaction scheme for the blockchain-based metaverse, ACM Trans. Multimedia Comput. Commun. Appl. 20 (7) (2024) 188:1–188:23.
[16] T. Jiang, X. Chen, J. Ma, Public integrity auditing for shared dynamic cloud data with group user revocation, IEEE Trans. Comput. 65 (8) (2016) 2363–2373.
[17] H. Yan, W. Gui, Efficient identity-based public integrity auditing of shared data in cloud storage with user privacy preserving, IEEE Access 9 (2021) 45822–45831.
[18] J. Yuan, S. Yu, Public integrity auditing for dynamic data sharing with multiuser modification, IEEE Trans. Inf. Forensics Secur. 10 (8) (2015) 1717–1726.
[19] B. Wang, B. Li, H. Li, Panda: Public auditing for shared data with efficient user revocation in the cloud, IEEE Trans. Serv. Comput. 8 (1) (2015) 92–106.
[20] H. Duan, Y. Du, L. Zheng, C. Wang, M.H. Au, Q. Wang, Towards practical auditing of dynamic data in decentralized storage, IEEE Trans. Dependable Secur. Comput. 20 (1) (2023) 708–723.
[21] H. Tian, F. Nan, H. Jiang, C. Chang, J. Ning, Y. Huang, Public auditing for shared cloud data with efficient and secure group management, Inf. Sci. 472 (2019) 107–125.
[22] R. Rabaninejad, M. Ahmadian-Attari, M.R. Asaar, M.R. Aref, A lightweight auditing service for shared data with secure user revocation in cloud storage, IEEE Trans. Serv. Comput. 15 (1) (2022) 1–15.
[23] D. Kim, K.S. Kim, Privacy-preserving public auditing for shared cloud data with secure group management, IEEE Access 10 (2022) 44212–44223.
[24] J. Li, H. Yan, Y. Zhang, Identity-based privacy preserving remote data integrity checking for cloud storage, IEEE Syst. J. 15 (1) (2021) 577–585.
[25] Y. Zhang, S. Wang, X. Qu, C. Li, E. Xu, Toward secure and efficient collaborative cached data auditing for distributed fog computing, IEEE Internet Things J. 10 (23) (2023) 20941–20954.
[26] Y. Tian, H. Tan, J. Shen, P. Vijayakumar, B.B. Gupta, V. Arya, Efficient identity-based multi-copy data sharing auditing scheme with decentralized trust management, Inf. Sci. 644 (2023) 119255.
[27] H. Yan, J. Li, J. Han, Y. Zhang, A novel efficient remote data possession checking protocol in cloud storage, IEEE Trans. Inf. Forensics Secur. 12 (1) (2017) 78–88.
[28] Y. Yu, Y. Li, J. Ni, G. Yang, Y. Mu, W. Susilo, Comments on "public integrity auditing for dynamic data sharing with multiuser modification", IEEE Trans. Inf. Forensics Secur. 11 (3) (2016) 658–659.
[29] K. He, J. Chen, Q. Yuan, S. Ji, D. He, R. Du, Dynamic group-oriented provable data possession in the cloud, IEEE Trans. Dependable Secur. Comput. 18 (3) (2021) 1394–1408.
[30] Y. Li, Y. Li, K. Zhang, Y. Ding, Public integrity auditing for dynamic group cooperation files with efficient user revocation, Comput. Stand. Interfaces 83 (2023) 103641.
[31] F. Tang, Y. Li, Y. Zhang, W. Susilo, B. Li, Real-time privacy-preserved auditing for shared outsourced data, Comput. Stand. Interfaces 92 (2025) 103927.
[32] R. Ding, Y. Xu, H. Zhong, J. Cui, K. Sha, Towards fully anonymous integrity checking and reliability authentication for cloud data sharing, IEEE Trans. Serv. Comput. 16 (5) (2023) 3782–3795.
[33] J. Chen, T. Zhou, S. Ji, H. Tan, W. Zheng, Efficient public auditing scheme for non-administrator group with secure user revocation, J. Inf. Secur. Appl. 80 (2024) 103676.
[34] H. Tian, N. Gan, F. Peng, H. Quan, C. Chang, A.V. Vasilakos, Smart contract-based public integrity auditing for cloud storage against malicious auditors, Future Gener. Comput. Syst. 166 (2025) 107709.
[35] B. Wang, B. Li, H. Li, Knox: Privacy-preserving auditing for shared data with large groups in the cloud, in: F. Bao, P. Samarati, J. Zhou (Eds.), Proceedings of ACNS, Vol. 7341, 2012, pp. 507–525.
[36] Y. Yu, J. Ni, M.H. Au, Y. Mu, B. Wang, H. Li, Comments on a public auditing mechanism for shared cloud data service, IEEE Trans. Serv. Comput. 8 (6) (2015) 998–999.
[37] J. Li, H. Yan, Y. Zhang, Certificateless public integrity checking of group shared data on cloud storage, IEEE Trans. Serv. Comput. 14 (1) (2021) 71–81.
[38] Y. Miao, Q. Huang, M. Xiao, H. Li, Decentralized and privacy-preserving public auditing for cloud storage based on blockchain, IEEE Access 8 (2020) 139813–139826.
[39] J. Li, H. Yan, Y. Zhang, Efficient identity-based provable multi-copy data possession in multi-cloud storage, IEEE Trans. Cloud Comput. 10 (1) (2022) 356–365.
[40] G. Chen, R. Hao, M. Yang, Popularity-based multiple-replica cloud storage integrity auditing for big data, Future Gener. Comput. Syst. 163 (2025) 107534.
[41] S. Gorbunov, L. Reyzin, H. Wee, Z. Zhang, Pointproofs: Aggregating proofs for multiple vector commitments, in: J. Ligatti, X. Ou, J. Katz, G. Vigna (Eds.), Proceedings of CCS, 2020, pp. 2007–2023.
[42] A. Miller, M. Hicks, J. Katz, E. Shi, Authenticated data structures, generically, in: S. Jagannathan, P. Sewell (Eds.), Proceedings of POPL, ACM, 2014, pp. 411–424.
[43] M. Bellare, P. Rogaway, Random oracles are practical: A paradigm for designing efficient protocols, in: D.E. Denning, R. Pyle, R. Ganesan, R.S. Sandhu, V. Ashby (Eds.), Proceedings of CCS, ACM, 1993, pp. 62–73.
[44] G. Fuchsbauer, E. Kiltz, J. Loss, The algebraic group model and its applications, in: H. Shacham, A. Boldyreva (Eds.), Proceedings of CRYPTO, Vol. 10992, Springer, 2018, pp. 33–62.
[45] D. Boneh, B. Lynn, H. Shacham, Short signatures from the weil pairing, in: C. Boyd (Ed.), Proceedings of ASIACRYPT, Vol. 2248, 2001, pp. 514–532.